UNIVERSIDAD NACIONAL DEL ALTIPLANO

FACULTAD DE INGENIERÍA MECÁNICA ELÉCTRICA, ELECTRÓNICA Y SISTEMAS.

ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS

”VISION ARTIFICIAL”

“INFORME DETECCIÓN DE OBJETOS CON FASTER R-CNN Y YOLO”

DOCENTE:

ING. FERNANDEZ CHAMBI MAYENKA

PRESENTAN:

- ANCCO GARCIA JOSE MANUEL
- MESTAS CALSIN ALEX CRISTIAN
- MESSCO CONDORI BENIGNO SAUL
- QUISPE SONCCO WILLEBELDO FERMIN
- TICONA ERQUINIGO JHOEL JOVANI

SEMESTRE: X **GRUPO:** B

PUNO-PERÚ

11 DE NOVIEMBRE 2026

Contenido

Introducción 3

2. Metodología 4

2.1 Modelos utilizados 4

2.2 Datos y entorno de prueba 5

2.3 Métricas de evaluación 5

3. Detección de objetos en imágenes 7

3.1 Faster R-CNN en imágenes 7

3.2 YOLO en imágenes 7

3.3 Comparación de resultados en imágenes 8

4. Detección de objetos en video y tiempo real 11

4.1 Faster R-CNN en video 11

4.2 YOLO en video 12

4.3 Comparación de resultados en video 13

5. Comportamiento en imágenes y en video según las métricas 14

5.1 Exactitud frente a velocidad 15

5.2 La ventaja en imágenes no se traslada al video 15

5.3 Efecto de la arquitectura en la latencia 15

5.4 Confianza y estabilidad temporal 16

5.5 Validez de la comparación 16

6. Conclusiones 16

Referencias 17

Introducción

La detección de objetos es una de las tareas centrales de la visión artificial. Consiste en localizar, mediante cajas delimitadoras, cada objeto presente en una imagen y asignarle una clase con un nivel de confianza. A diferencia de la clasificación, que asigna una sola etiqueta a toda la imagen, la detección debe resolver al mismo tiempo qué objetos hay y dónde se encuentran, lo que la hace indispensable en aplicaciones como la videovigilancia, la conducción autónoma, el conteo de personas o la inspección industrial. Entre los detectores basados en redes neuronales convolucionales destacan dos familias: los detectores de dos etapas, representados por Faster R-CNN, y los detectores de una etapa, representados por YOLO (*You Only Look Once*).

Faster R-CNN primero genera regiones candidatas mediante una red de propuestas de regiones (RPN) y luego clasifica y ajusta cada una de ellas, lo que favorece la precisión a costa de un mayor tiempo de cómputo. YOLO, en cambio, predice todas las cajas y clases en una sola pasada de la red sobre una rejilla que cubre la imagen, con lo que prioriza la velocidad. Esta diferencia de diseño hace suponer que cada familia se comporta de forma distinta según el escenario: en imágenes fijas suele importar más la exactitud de las detecciones, mientras que en video el factor determinante es procesar los fotogramas a la velocidad con que llegan.

El presente trabajo tiene como objetivo implementar y comparar ambos detectores en los dos escenarios. Se desarrollaron aplicaciones de detección con Faster R-CNN y con YOLO sobre imágenes fijas, evaluadas en el conjunto coco128 con las métricas estándar de COCO (mAP, precisión, recall y F1), y aplicaciones de detección en video y tiempo real con ambos modelos, evaluadas mediante la latencia, los fotogramas por segundo, el factor de tiempo real y la estabilidad temporal. A partir de estas métricas se explica el comportamiento de cada detector en imágenes y en video.

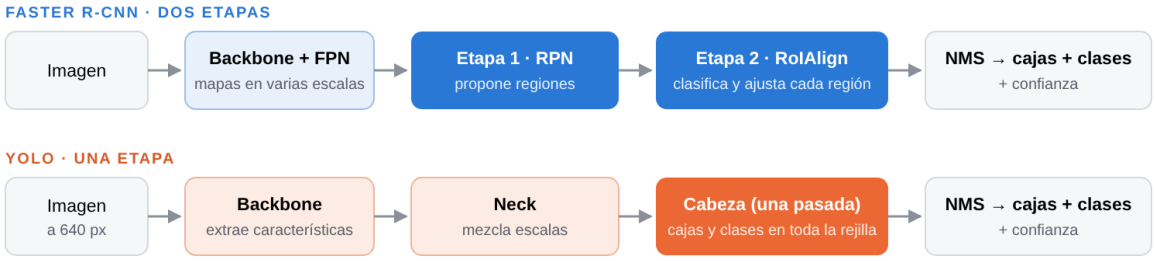
Para el desarrollo se utilizó el lenguaje Python con las bibliotecas OpenCV, PyTorch/torchvision, ONNX Runtime y Ultralytics, y la herramienta *pycocotools* para el cálculo de las métricas. Todas las pruebas se realizaron en una misma computadora con procesador AMD y sistema operativo Windows, sin unidad de procesamiento gráfico (GPU), de modo que las diferencias de tiempo reflejan únicamente el costo computacional de cada modelo.

2. Metodología

2.1 Modelos utilizados

Se emplearon modelos preentrenados en el conjunto COCO, que comprende 80 clases de objetos, sin realizar un reentrenamiento. De esta forma se compara el comportamiento de cada arquitectura en igualdad de condiciones. La Figura 1 resume el flujo de procesamiento de ambos detectores.

Figura 1. *Flujo de procesamiento de Faster R-CNN (dos etapas) y de YOLO (una etapa)*



Fuente: elaboración propia.

En Faster R-CNN, la imagen atraviesa una red troncal (*backbone*) con una pirámide de características (FPN) que genera mapas en varias escalas. Sobre ellos, la RPN propone regiones que probablemente contienen un objeto; en la segunda etapa, la operación RoIAlign extrae las características de cada región y una cabeza de clasificación asigna la clase y corrige la caja. En YOLO, la imagen se redimensiona a 640 píxeles y atraviesa la red troncal, un cuello (*neck*) que combina escalas y una cabeza que predice directamente cajas y clases en cada celda de la rejilla. En ambos casos, la supresión de no máximos (NMS) elimina las cajas duplicadas.

La Tabla 1 detalla las versiones utilizadas en cada parte del trabajo. Para imágenes se empleó Faster R-CNN con red troncal ResNet-50 y FPN, ejecutado en formato ONNX, y YOLOv8n, la versión nano de YOLOv8, con 3,2 millones de parámetros. Para video se mantuvo YOLOv8n, pero Faster R-CNN se reemplazó por su variante ligera MobileNetV3-Large-320-FPN, ya que el modelo con ResNet-50 resultaba demasiado lento para procesar video en CPU.

Tabla 1. *Modelos y parámetros utilizados en cada parte del trabajo*

Parte	Modelo	Clases	Umbral de confianza	Biblioteca
a	Faster R-CNN ResNet-50 + FPN	80 (COCO)	0,5	ONNX Runtime
b	YOLOv8n (entrada de 640 px)	80 (COCO)	0,5	Ultralytics
c	Faster R-CNN MobileNetV3-Large-320 + FPN (ancho de 640 px)	80 (COCO)	0,6	torchvision
d	YOLOv8n	10 (personas, vehículos y animales)	0,45	Ultralytics

Fuente: elaboración propia.

2.2 Datos y entorno de prueba

Para las imágenes se utilizó *coco128*, un subconjunto de 128 imágenes del conjunto COCO que contiene 929 objetos etiquetados pertenecientes a 71 clases. Ambos modelos procesaron exactamente las mismas imágenes, y sus detecciones se evaluaron con el mismo protocolo mediante *pycocotools*, la herramienta oficial de evaluación de COCO.

Para el video se utilizó un mismo archivo en ambos modelos: una escena urbana con personas, automóviles y motocicletas, de 1304 fotogramas a 29,97 fotogramas por segundo (FPS). Como el video no dispone de etiquetas reales, en este escenario no es posible calcular el mAP, por lo que la evaluación se centra en la velocidad y en la consistencia de las detecciones. En todos los casos se excluyeron los cinco primeros fotogramas del cálculo de tiempos, porque incluyen la carga del modelo en memoria.

2.3 Métricas de evaluación

La evaluación en imágenes se basa en la intersección sobre la unión (IoU), que mide el solape entre una caja predicha A y la caja real B, según la ecuación (1). Una detección se considera correcta (verdadero positivo, TP) cuando su IoU con un objeto real de la misma clase supera un valor mínimo; en caso contrario es un falso positivo (FP), y cada objeto real no detectado es un falso negativo (FN).

$$\mathrm{IoU} = \frac{\text{área}\,(A \cap B)}{\text{área}\,(A \cup B)} \tag{1}$$

A partir de estos conteos se calculan la precisión (P), que es la fracción de detecciones correctas; el recall (R), que es la fracción de objetos encontrados; y el F1, que es la media armónica de ambas, como se muestra en la ecuación (2). El umbral de confianza actúa como un filtro: al subirlo aumenta la precisión y disminuye el recall.

$$P = \frac{TP}{TP + FP} \qquad R = \frac{TP}{TP + FN} \qquad F1 = \frac{2 \cdot P \cdot R}{P + R} \tag{2}$$

La métrica principal es el mAP@[.5:.95] de COCO, que promedia la precisión media (AP) de todas las clases para diez exigencias de solape, con IoU de 0,50 a 0,95. Como esta métrica recorre todos los niveles de confianza, no depende del umbral elegido y permite una comparación justa entre modelos. En video, la velocidad se expresa mediante el factor de tiempo real de la ecuación (3), donde los FPS del modelo se obtienen de la latencia media según la ecuación (4); un valor del factor mayor o igual a 1 indica que el modelo procesa los fotogramas al ritmo con que llegan.

$$\text{Factor de tiempo real} = \frac{\text{FPS del modelo}}{\text{FPS del video}} \tag{3}$$

$$\text{FPS del modelo} = \frac{1000}{\text{latencia media (ms)}} \tag{4}$$

La Tabla 2 resume las métricas empleadas en cada escenario.

Tabla 2. *Métricas de evaluación utilizadas*

Escenario	Métrica	Qué indica
Imagen	mAP@[.5:.95]	Precisión media con IoU de 0,50 a 0,95; métrica principal de COCO e independiente del umbral
Imagen	mAP@.5 y mAP@.75	Precisión media con una exigencia de solape baja (0,5) o alta (0,75)
Imagen	AP por tamaño	AP en objetos pequeños (< 32^2 px), medianos y grandes (> 96^2 px)
Imagen	AR@100	Recall máximo con hasta 100 detecciones por imagen
Imagen	Precisión, recall y F1	Calidad de las detecciones con conf ≥ 0,5 e IoU ≥ 0,5, y barrido de umbrales de 0,1 a 0,9
Ambos	Latencia y FPS	Tiempo por imagen o por fotograma; para 30 FPS se requieren 33 ms o menos
Video	Factor de tiempo real	FPS del modelo entre FPS del video; con un valor ≥ 1 el modelo alcanza a la cámara
Video	Estabilidad temporal	Fracción de detecciones que se mantienen en el fotograma siguiente (misma clase, IoU ≥ 0,5)
Video	Variación del conteo	Cambio medio del número de objetos entre fotogramas consecutivos

Fuente: elaboración propia.

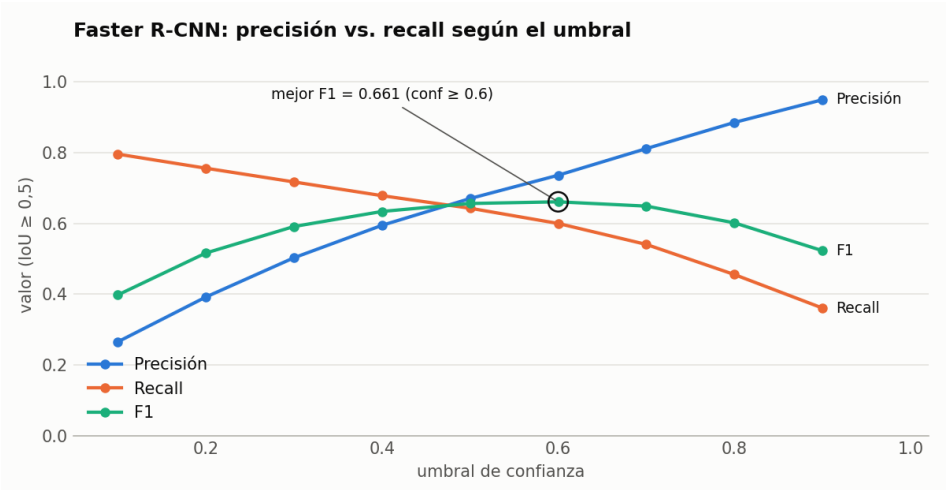
3. Detección de objetos en imágenes

3.1 Faster R-CNN en imágenes

La aplicación de Faster R-CNN procesa cada imagen, dibuja las detecciones con una confianza mayor o igual a 0,5 y guarda las métricas en un archivo JSON. Sobre las 128 imágenes, el modelo obtuvo un mAP@[.5:.95] de 0,431, un mAP@.5 de 0,666 y un recall máximo (AR@100) de 0,529. Con el umbral de 0,5 realizó 891 detecciones, de las cuales 597 fueron correctas, lo que equivale a una precisión de 0,670 y un recall de 0,643 sobre los 929 objetos reales. El tiempo medio de procesamiento fue de 2173 ms por imagen (0,46 FPS), con un percentil 95 de 2609 ms.

La Figura 2 muestra el efecto del umbral de confianza. Al subir el umbral, la precisión aumenta y el recall disminuye, porque se descartan detecciones dudosas, pero también algunos objetos reales. El mejor equilibrio entre ambas, medido por el F1, se alcanza con un umbral de 0,6 (F1 = 0,661), muy cerca del valor de 0,5 utilizado, lo que indica que las puntuaciones de confianza de Faster R-CNN están bien calibradas.

Figura 2. *Precisión, recall y F1 de Faster R-CNN en función del umbral de confianza*



Fuente: elaboración propia.

3.2 YOLO en imágenes

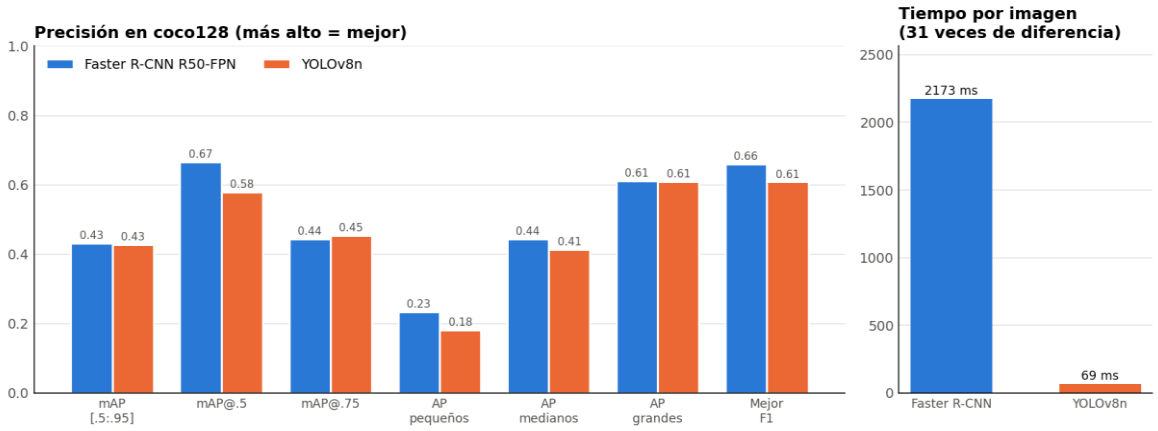
YOLOv8n obtuvo un mAP@[.5:.95] de 0,427, prácticamente igual al de Faster R-CNN, con un mAP@.5 de 0,580 y un AR@100 de 0,494. Su tiempo medio fue de 69 ms por imagen (14,5 FPS), es decir, unas 31 veces menor que el de Faster R-CNN; en una segunda ejecución el tiempo medio fue de 67 ms, lo que corresponde a la variación normal entre corridas.

Con el umbral de 0,5, sin embargo, YOLO realizó solo 372 detecciones, de las cuales 332 fueron correctas: una precisión muy alta (0,892), pero un recall bajo (0,357). Este resultado no significa que el modelo detecte menos objetos, sino que asigna puntuaciones de confianza más bajas que Faster R-CNN; de hecho, su mejor F1 (0,609) se obtiene con un umbral de 0,2. En consecuencia, un mismo umbral fijo no tiene el mismo significado en ambos modelos, y la comparación justa debe hacerse con el mAP, que no depende del umbral.

3.3 Comparación de resultados en imágenes

La Figura 3 y la Tabla 3 comparan ambos modelos bajo el mismo protocolo de evaluación.

Figura 3. *Métricas de precisión (izquierda) y tiempo medio por imagen (derecha) en coco128*



Fuente: elaboración propia.

Tabla 3. *Comparación de Faster R-CNN y YOLOv8n en imágenes*

Métrica	Faster R-CNN R50-FPN	YOLOv8n	Mejor resultado
mAP@[.5:.95]	0,431	0,427	Empate técnico
mAP@.5	**0,666**	0,580	Faster R-CNN
mAP@.75	0,445	**0,454**	YOLOv8n
AP en objetos pequeños	**0,235**	0,181	Faster R-CNN (+30 %)
AP en objetos medianos	**0,443**	0,414	Faster R-CNN
AP en objetos grandes	0,612	0,610	Empate técnico
AR@100	**0,529**	0,494	Faster R-CNN
Precisión / recall (conf 0,5)	0,670 / 0,643	0,892 / 0,357	Depende del umbral
Mejor F1 (umbral)	**0,661 (0,6)**	0,609 (0,2)	Faster R-CNN
Latencia media / FPS	2173 ms / 0,46	**69 ms / 14,5**	YOLOv8n (31 veces más rápido)

Fuente: elaboración propia.

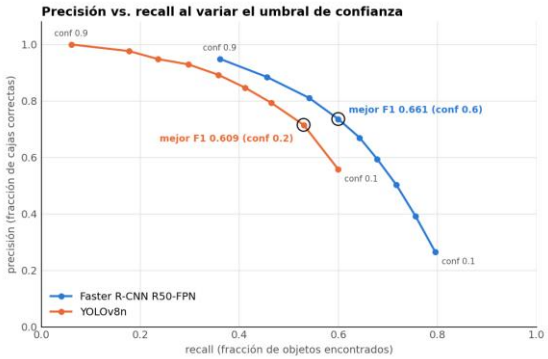
El empate en el mAP@[.5:.95] (0,431 frente a 0,427) oculta dos comportamientos distintos. Faster R-CNN supera claramente a YOLO en mAP@.5 (0,666 frente a 0,580); es decir, encuentra más objetos cuando se exige un solape moderado. YOLO, en cambio, obtiene un mAP@.75 ligeramente mayor (0,454 frente a 0,445), lo que indica que sus cajas, cuando acierta, están algo mejor ajustadas al objeto. Al promediar todos los niveles de IoU, ambas ventajas se compensan.

La mayor diferencia se presenta en los objetos pequeños, donde Faster R-CNN obtiene un AP de 0,235 frente a 0,181 de YOLO, un 30 % más, y un recall de 0,319 frente a 0,215. Esto se explica por tres características de Faster R-CNN: la FPN combina mapas de varias escalas, el modelo trabaja con una resolución de entrada mayor y la segunda etapa analiza en detalle cada región propuesta. YOLOv8n, al ser la versión más pequeña de su familia, pierde detalle en los objetos que ocupan pocos píxeles. En los objetos grandes ambos modelos rinden igual (0,612 frente a 0,610).

La Figura 4 muestra la curva de precisión frente a recall al variar el umbral de 0,1 a 0,9. La curva de Faster R-CNN se ubica por encima y a la derecha de la de YOLO en casi todo el recorrido, de modo que, con el umbral adecuado, ofrece más precisión para un mismo recall. YOLO no supera un recall de 0,60 ni siquiera con un umbral de 0,1, aunque con umbrales altos casi no comete errores (precisión de 0,93 a 1,00). En la práctica, si se desea que YOLO encuentre la mayor cantidad de objetos, su umbral debe reducirse a valores entre 0,2 y 0,3.

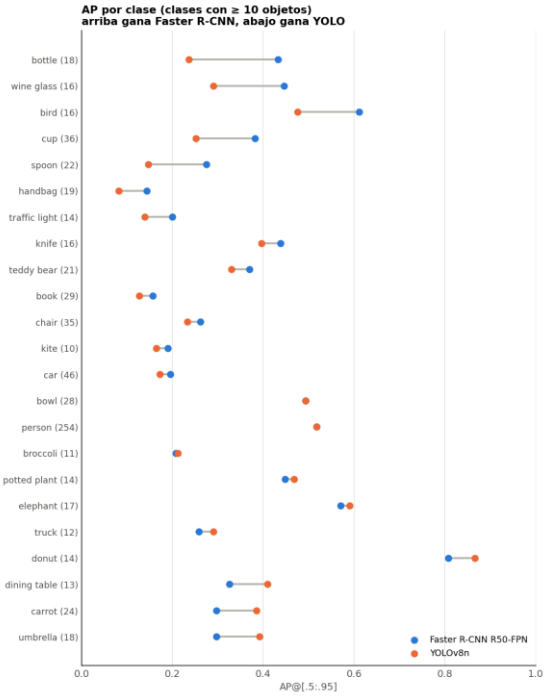
La Figura 5 compara el AP por clase. De las 23 clases con al menos 10 objetos, Faster R-CNN obtiene mejor resultado en 13, YOLO en 8 y en 2 se produce un empate. Las mayores ventajas de Faster R-CNN corresponden a objetos pequeños o delgados, como botellas (0,43 frente a 0,24), copas (0,45 frente a 0,29), aves (0,61 frente a 0,48), tazas (0,38 frente a 0,25) y cucharas (0,28 frente a 0,15). YOLO obtiene mejores resultados en clases como paraguas (0,39 frente a 0,30), mesas (0,41 frente a 0,33), zanahorias (0,39 frente a 0,30) y donas (0,87 frente a 0,81). En la clase persona, la más frecuente con 254 objetos, ambos modelos empatan exactamente (0,517).

Figura 4. *Curva precisión-recall de ambos modelos al variar el umbral de confianza*



Fuente: elaboración propia.

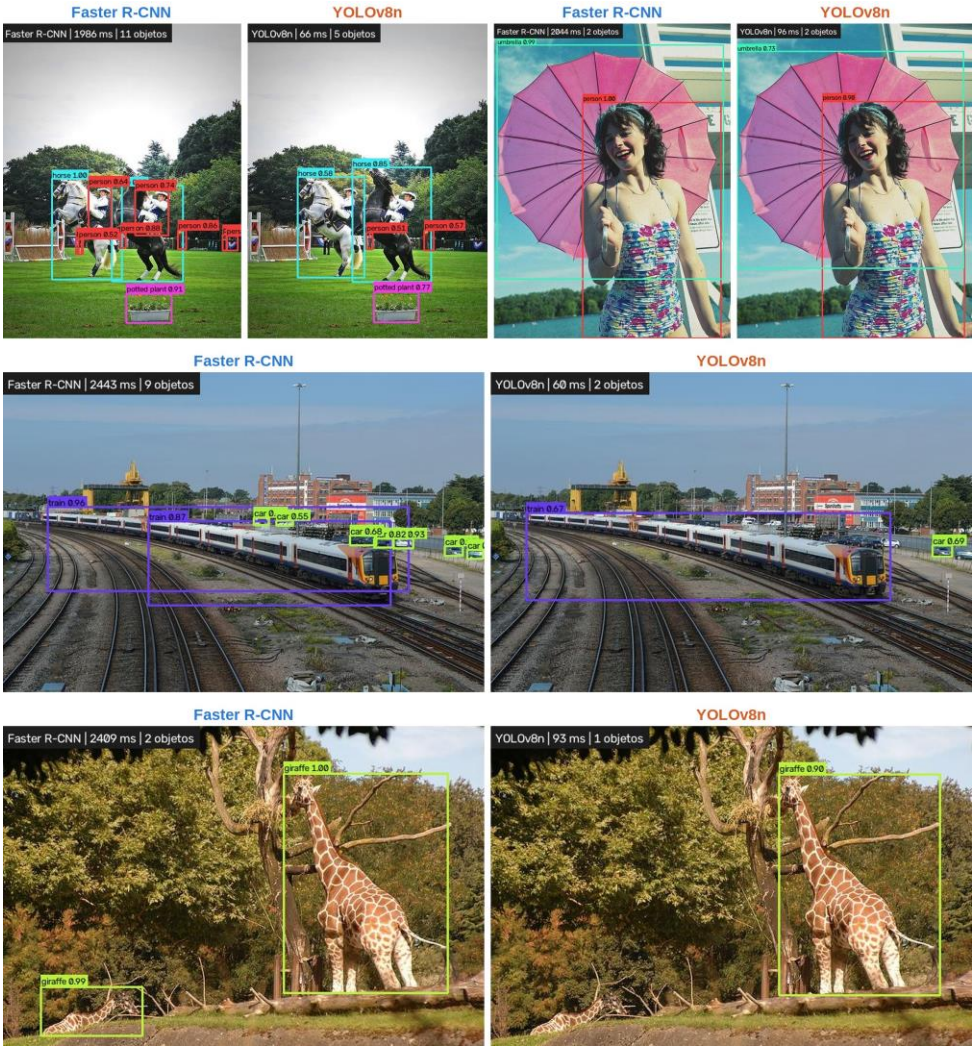
Figura 5. *AP@[.5:.95] por clase en las clases con 10 o más objetos*



Fuente: elaboración propia.

La Figura 6 presenta ejemplos de detección sobre las mismas imágenes. En la escena de caballos y jinetes, Faster R-CNN detecta 11 objetos y YOLO, 5; en la estación de tren, Faster R-CNN identifica 9 objetos, entre ellos vehículos lejanos, mientras que YOLO identifica 2; y en la imagen de jirafas, solo Faster R-CNN detecta la jirafa pequeña ubicada en el fondo. En escenas simples, como la del paraguas, ambos modelos producen las mismas cajas, aunque Faster R-CNN lo hace con mayor confianza (0,99 y 1,00 frente a 0,73 y 0,90) y con un tiempo de 2044 ms frente a 96 ms de YOLO.

Figura 6. *Ejemplos de detección con conf ≥ 0,5 sobre las mismas imágenes*



Fuente: elaboración propia.

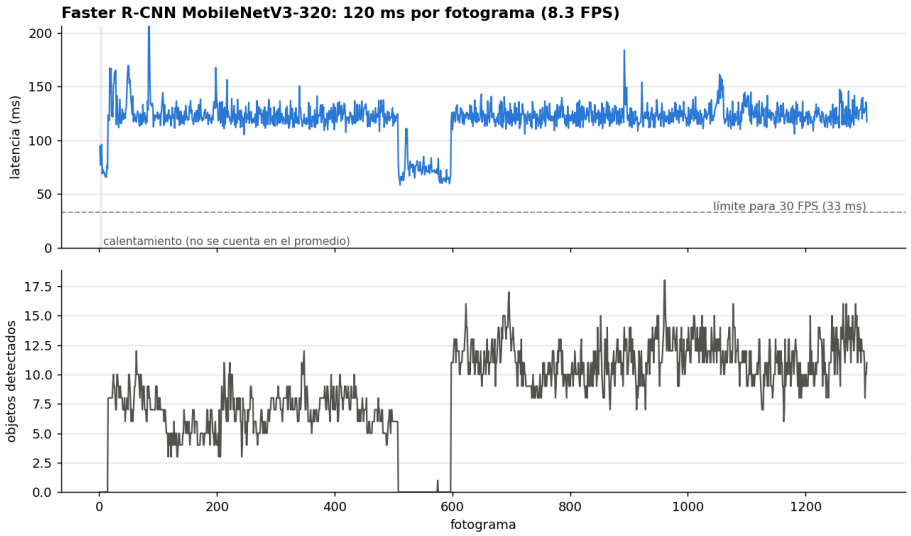
4. Detección de objetos en video y tiempo real

4.1 Faster R-CNN en video

La aplicación de video lee el archivo fotograma por fotograma, lo redimensiona a un ancho de 640 píxeles, ejecuta el detector, dibuja las cajas y muestra el resultado en una ventana; además, registra el tiempo y las detecciones de cada fotograma. Con Faster R-CNN MobileNetV3-320 y un umbral de 0,6, la latencia media fue de 120,0 ms por fotograma (8,33 FPS), con un percentil 95 de 137,7 ms. El factor de tiempo real fue de 0,28, lo que significa que, con una cámara en vivo, el modelo solo alcanzaría a procesar uno de cada cuatro fotogramas (72,2 % de pérdida estimada). El procesamiento completo del video tomó 171,6 s.

La Figura 7 muestra la latencia y el número de objetos de cada fotograma. La latencia nunca desciende por debajo del límite de 33 ms necesario para 30 FPS. Se observa, además, que en el tramo del video sin objetos (aproximadamente entre los fotogramas 505 y 600) la latencia baja a unos 72 ms, mientras que en los fotogramas con objetos se mantiene alrededor de 124 ms.

Figura 7. *Latencia y objetos detectados por fotograma con Faster R-CNN MobileNetV3-320*



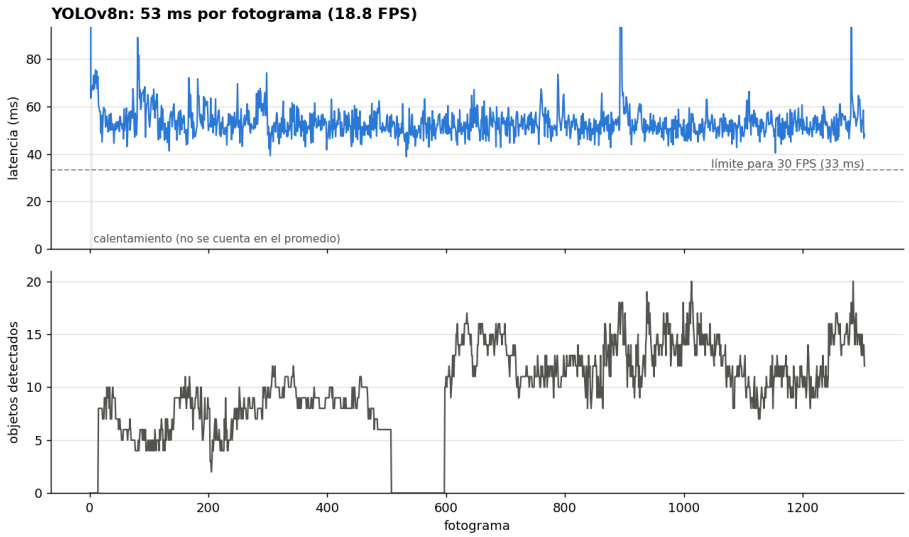
Fuente: elaboración propia.

4.2 YOLO en video

La aplicación de YOLO permite elegir entre la cámara web, un archivo de video o una imagen, y se restringe a 10 clases de interés: personas, automóviles, motocicletas, camiones y seis clases de animales. Con un umbral de 0,45, YOLOv8n alcanzó una latencia media de 53,2 ms por fotograma (18,81 FPS) y un percentil 95 de 63,2 ms. Su factor de tiempo real fue de 0,63, de modo que procesaría aproximadamente dos de cada tres fotogramas de una cámara en vivo (37,3 % de pérdida estimada). El procesamiento completo tomó 84,2 s.

Como se observa en la Figura 8, la latencia de YOLO es prácticamente constante, alrededor de 53 ms, y no cambia en el tramo sin objetos. El número de objetos detectados sigue la misma forma que en Faster R-CNN: entre 5 y 10 objetos al inicio, un tramo vacío y una escena más concurrida a partir del fotograma 600, lo que confirma que ambos modelos responden a los mismos cambios de la escena.

Figura 8. *Latencia y objetos detectados por fotograma con YOLOv8n*

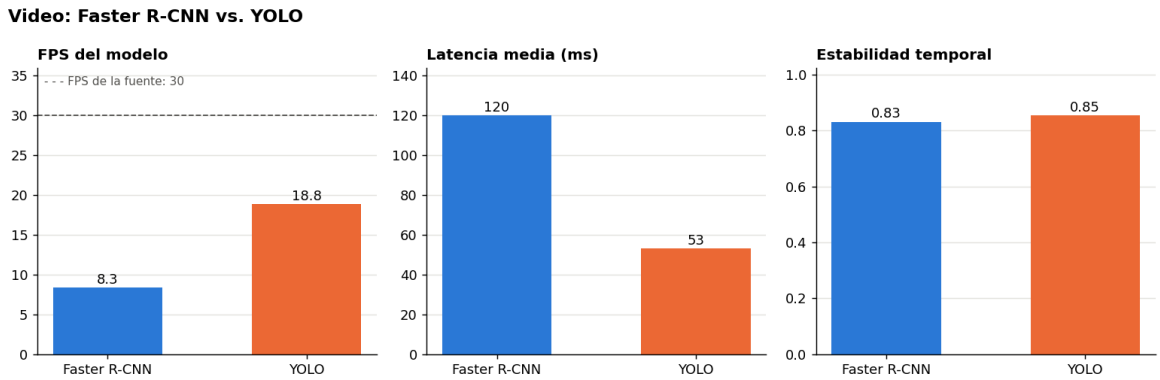


Fuente: elaboración propia.

4.3 Comparación de resultados en video

La Figura 9 y la Tabla 4 comparan ambos modelos sobre el mismo video y en la misma computadora.

Figura 9. *FPS del modelo frente a los 30 FPS del video, latencia media y estabilidad temporal*



Fuente: elaboración propia.

Tabla 4. *Comparación de Faster R-CNN y YOLOv8n en video*

Métrica	Faster R-CNN MobileNetV3-320	YOLOv8n
Umbral de confianza	0,60	0,45
Latencia media / mediana / p95	120,0 / 121,7 / 137,7 ms	**53,2 / 52,5 / 63,2 ms**
FPS del modelo / del bucle completo	8,33 / 7,59	**18,81 / 15,51**
Factor de tiempo real	0,28	**0,63**
Fotogramas perdidos en vivo (estimado)	72,2 %	**37,3 %**
Objetos por fotograma (máximo)	8,71 (18)	9,73 (20)
Confianza media	**0,856**	0,692
Estabilidad temporal	0,831	**0,854**
Variación del conteo	0,94	**0,81**
Tiempo total (1304 fotogramas)	171,6 s	**84,2 s**
Clases más detectadas	personas 9936; motos 642; autos 427; buses 123	personas 11 792; autos 426; motos 404; camiones 68

Fuente: elaboración propia.

Ninguno de los dos modelos alcanza el tiempo real en CPU, ya que para 30 FPS el presupuesto es de 33 ms por fotograma. YOLO es 2,3 veces más rápido que Faster R-CNN y, además, más regular: su percentil 95 (63 ms) está muy cerca de su media, por lo que casi no presenta fotogramas lentos.

En cuanto a la consistencia, alrededor del 15 % de las detecciones de ambos modelos «parpadea», es decir, desaparece de un fotograma al siguiente. YOLO es ligeramente más estable (0,854 frente a 0,831) y su conteo de objetos varía menos entre fotogramas (0,81 frente a 0,94), pese a utilizar un umbral más bajo. Las detecciones coinciden en las clases que ambos comparten; por ejemplo, el número de detecciones de automóviles es prácticamente igual (427 y 426). Las diferencias en otras clases se deben a que YOLO solo considera 10 clases: no incluye, por ejemplo, la clase autobús, que Faster R-CNN detectó 123 veces.

5. Comportamiento en imágenes y en video según las métricas

La Tabla 5 resume cómo cambia el comportamiento de cada detector al pasar de imágenes fijas a video. A continuación se explican las diferencias observadas a partir de las métricas.

Tabla 5. *Comportamiento de los detectores en imágenes y en video*

Aspecto	Imágenes	Video
Qué se mide	Precisión (mAP, P, R) frente a etiquetas reales	Velocidad y consistencia (sin etiquetas)
Faster R-CNN	ResNet-50 FPN: 2173 ms; mAP 0,431	MobileNetV3-320: 120 ms (18 veces más rápido), con un modelo menos preciso
YOLOv8n	69 ms	53 ms (mismo modelo)
Detecta más objetos	Faster R-CNN (891 frente a 372 con conf 0,5)	YOLO (9,7 frente a 8,7 objetos por fotograma)
Confianza media	Faster R-CNN 0,806; YOLO 0,751	Faster R-CNN 0,856; YOLO 0,692
Latencia según el contenido	No aplica	Faster R-CNN: 72 ms sin objetos y 124 ms con objetos; YOLO: 53 ms en ambos casos

Fuente: elaboración propia.

5.1 Exactitud frente a velocidad

En imágenes fijas, el criterio principal es la exactitud: que el modelo tarde dos segundos por imagen no impide su uso. En video, en cambio, el criterio determinante es la velocidad, porque los fotogramas llegan cada 33 ms. Con 2173 ms por fotograma, Faster R-CNN con ResNet-50 procesaría menos del 2 % de los fotogramas de un video a 30 FPS; por ello, la parte de video tuvo que emplear la versión MobileNetV3-320. Esta versión es 18 veces más rápida, pero su mAP oficial en COCO val es de 22,8, frente a 37,0 del modelo con ResNet-50 de torchvision. YOLOv8n, por el contrario, pudo emplearse sin cambios en ambos escenarios.

5.2 La ventaja en imágenes no se traslada al video

En imágenes, Faster R-CNN detectó 2,4 veces más objetos que YOLO con el mismo umbral; en video la situación se invierte y YOLO detecta más objetos por fotograma (9,7 frente a 8,7). La causa es que el modelo de video es distinto: trabaja a 320 píxeles, lo que lo hace más débil con los objetos pequeños, y utiliza un umbral más alto (0,6 frente a 0,45). Por lo tanto, la precisión medida en imágenes es válida para el modelo concreto evaluado y no para toda su familia de detectores.

5.3 Efecto de la arquitectura en la latencia

La diferencia entre un detector de dos etapas y uno de una etapa se refleja en el comportamiento de la latencia. En Faster R-CNN, la segunda etapa trabaja sobre las regiones que propone la RPN; cuando el fotograma no contiene objetos hay menos regiones relevantes

y el trabajo disminuye, por lo que la latencia baja de 124 ms a 72 ms. YOLO, en cambio, realiza una única pasada fija sobre toda la imagen y tarda 53 ms con o sin objetos. Este tiempo predecible es una ventaja importante en las aplicaciones de tiempo real, donde interesa garantizar una frecuencia de procesamiento estable.

5.4 Confianza y estabilidad temporal

La confianza media de Faster R-CNN aumenta en video (de 0,806 a 0,856) porque su umbral se elevó a 0,6, lo que descarta las detecciones dudosas. La de YOLO disminuye (de 0,751 a 0,692), porque su umbral bajó a 0,45 y porque los fotogramas de video presentan desenfoque por movimiento y compresión, a diferencia de las fotografías de COCO. Además, el video introduce una dimensión que no existe en las imágenes fijas: la estabilidad temporal. Un modelo con buen mAP puede producir detecciones que aparecen y desaparecen entre fotogramas; en este caso, ambos modelos son similares, con una ligera ventaja de YOLO.

5.5 Validez de la comparación

Las imágenes de *coco128* pertenecen al conjunto de entrenamiento de COCO, por lo que el mAP absoluto es optimista: los valores de referencia en COCO val son de aproximadamente 35 a 37 para Faster R-CNN con ResNet-50 FPN, según la implementación, y de 37,3 para YOLOv8n. La comparación entre ambos modelos sigue siendo válida, porque los dos fueron entrenados con los mismos datos. En video, los modelos no son exactamente equivalentes, ya que difieren en el umbral, en el número de clases y en la versión de Faster R-CNN; por ello, los conteos y la confianza deben interpretarse con cautela, mientras que la velocidad y la estabilidad son directamente comparables. Por último, los tiempos corresponden a una sola CPU y pueden variar ligeramente entre ejecuciones.

6. Conclusiones

El desarrollo de las aplicaciones permitió implementar y comparar un detector de dos etapas, Faster R-CNN, y uno de una etapa, YOLO, en imágenes fijas y en video. En imágenes, ambos modelos alcanzaron prácticamente la misma precisión global (mAP@[.5:.95] de 0,431 y 0,427), pero con perfiles distintos: Faster R-CNN encuentra más objetos, en especial los pequeños, mientras que YOLO es más conservador y ajusta ligeramente mejor las cajas.

La velocidad marca la principal diferencia entre ambos enfoques. YOLO fue 31 veces más rápido en imágenes y 2,3 veces más rápido en video, con una latencia estable que no depende del contenido de la escena. Faster R-CNN, en cambio, requirió sustituir su red troncal por una versión ligera para poder procesar video, lo que redujo su precisión. Aun así, en CPU ninguno de los dos modelos alcanzó los 30 FPS; para lograr tiempo real sería necesario utilizar una GPU, reducir la resolución, procesar uno de cada N fotogramas o exportar el modelo a formatos optimizados como ONNX u OpenVINO.

Se comprobó también que el umbral de confianza debe ajustarse a cada modelo: el mejor F1 se obtuvo con 0,6 en Faster R-CNN y con 0,2 en YOLO. Comparar ambos con un mismo umbral fijo favorece a uno u otro, por lo que el mAP constituye la métrica adecuada para una comparación neutral. Del mismo modo, las métricas de video deben complementar a las de imágenes, ya que la estabilidad temporal y el factor de tiempo real no pueden deducirse del mAP.

En conclusión, no existe un detector superior para todos los casos. Faster R-CNN es preferible cuando importa no perder objetos, como en el análisis de fotografías o en el conteo preciso fuera de línea, mientras que YOLO es la opción adecuada cuando importa el tiempo de respuesta, como en el procesamiento de video, las cámaras en vivo o los equipos sin GPU. La elección depende del escenario, del tamaño de los objetos de interés y de los recursos de cómputo disponibles.

Referencias

Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., … Adam, H. (2019). *Searching for MobileNetV3*. En Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). https://arxiv.org/abs/1905.02244

Jocher, G., Chaurasia, A. y Qiu, J. (2023). *Ultralytics YOLOv8* [Software]. https://docs.ultralytics.com/models/yolov8/

Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B. y Belongie, S. (2017). *Feature Pyramid Networks for Object Detection*. En Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). https://arxiv.org/abs/1612.03144

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P. y Zitnick, C. L. (2014). *Microsoft COCO: Common Objects in Context*. En European Conference on Computer Vision (ECCV). https://cocodataset.org

Redmon, J., Divvala, S., Girshick, R. y Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. En Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). https://arxiv.org/abs/1506.02640

Ren, S., He, K., Girshick, R. y Sun, J. (2015). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. En Advances in Neural Information Processing Systems (NeurIPS). https://arxiv.org/abs/1506.01497

TorchVision. (s. f.). *Models and pre-trained weights: Object detection*. PyTorch. https://docs.pytorch.org/vision/stable/models.html

Link de los códigos y soluciones:

https://github.com/alex25xo/01